LimeWire Studio Edition v2.0 — Operations Manual

Comprehensive guide to every feature, tab, and workflow

Version 2.0 — March 2026

Table of Contents

1. Getting Started
2. Tab 1: Search & Grab
3. Tab 2: Batch Download
4. Tab 3: Playlist
5. Tab 4: Converter
6. Tab 5: Player
7. Tab 6: Analyze
8. Tab 7: Stems
9. Tab 8: Effects
10. Tab 9: Discovery
11. Tab 10: Samples
12. Tab 11: Editor
13. Tab 12: Recorder
14. Tab 13: Spectrogram
15. Tab 14: Pitch/Time
16. Tab 15: Remixer
17. Tab 16: Batch Processor
18. Tab 17: Scheduler
19. Tab 18: History
20. Tab 19: Cover Art
21. Tab 20: Settings
22. Theme System
23. Keyboard Shortcuts
24. Plugin System
25. Security Architecture
26. Code Architecture
27. Troubleshooting

1. Getting Started

Installation

Prerequisites:

- Python 3.10 or higher (tested with 3.14)
- FFmpeg on your system PATH
- Windows 10/11, macOS, or Linux

Quick Install:

```
winget install ffmpeg
pip install -r requirements.txt
python LimeWire.py
```

You can also run via: `python -m limewire`

First Launch

On first launch, LimeWire creates config files in your home directory:

- `~/.limewire_settings.json` — Theme, download folder, proxy
- `~/.limewire_history.json` — Download log
- `~/.limewire_session.json` — Window state and last active tab

The status bar shows your module count (e.g., "5/13"). Click it to see what's installed/missing.

User Interface Overview

- **Logo Bar** — Gradient header with app name and version badge
- **Toolbar** — Emoji icon strip for quick tab navigation
- **Tab Notebook** — 20 tabs organized by workflow
- **Status Bar** — Module count, keyboard shortcuts hint, progress

2. Tab 1: Search & Grab

Purpose

Download audio or video from 1000+ websites using a single URL.

How to Use

1. **Paste a URL** into the Search field. Source auto-detected (YouTube, SoundCloud, etc.).
2. **Choose mode** — Audio or Video. Audio shows formats like MP3, WAV, FLAC. Video shows MP4, MKV, etc.
3. **Select format** from the dropdown.
4. **Click Download** — Progress bar shows percentage and speed.
5. **After download** — File Info panel shows Title, Artist, Duration, Source. Quick-action buttons jump to Analyze or Stems.

Advanced Features

- **Clipboard Watch** — Auto-detect URLs copied to clipboard
- **Proxy/Rate Limit** — HTTP/SOCKS proxy and speed cap
- **Subtitles** — Check "Download subtitles" for video
- **SoundCloud/Bandcamp Search** — Type `sc:query` or `bc:query`

How the Code Works

`SearchPage` uses yt-dlp in a background thread. `_grab()` constructs yt-dlp options and launches `_do_grab()` in a thread. Progress hooks update the UI via `widget.after(0, callback)`. Downloaded audio gets ID3 tags via mutagen. URL validated by `is_url()` — checks scheme (http/https only), length (max 500), and pattern matching.

3. Tab 2: Batch Download

Purpose

Queue multiple URLs and download them in parallel.

How to Use

1. **Add URLs** one at a time or paste in bulk.
2. **Configure** — Format, quality, thread count (1-5 workers).
3. **Click Start** — Downloads run in parallel via ThreadPoolExecutor.
4. **Log display** shows colored output: green=success, yellow=warning, red=error.

Features

- **Persistent Queue** — Saved to `~/.limewire_queue.json`, survives restarts
- **Skip Already Downloaded** — Archive file tracking
- **Parallel Workers** — 1-5 configurable threads

How the Code Works

`DownloadPage` uses `ThreadPoolExecutor(max_workers=N)`. Each URL runs `_dl()` in its own thread. Log updates via `widget.after(0, ...)` for thread safety. Queue persists with `save_json()`.

4. Tab 3: Playlist

Purpose

Fetch YouTube playlists, select specific tracks, download them.

How to Use

1. **Paste playlist URL** and click Fetch.
2. **Select tracks** — Checkboxes for each track. All/None buttons for bulk selection.
3. **Download Selected** — Only checked tracks are downloaded.
4. **Retry Failed** — Re-attempt any failed downloads.

How the Code Works

Uses yt-dlp's `extract_flat=True` for fast playlist enumeration. Checkbutton widgets created dynamically. Downloads run in a background thread with per-URL error tracking.

5. Tab 4: Converter

Purpose

Convert audio/video files between formats using FFmpeg.

How to Use

1. **Add files** via file dialog.
2. **Select output format** — MP3, WAV, FLAC, AAC, OGG, M4A, MP4, MKV.
3. **Set bitrate** — 320k, 256k, 192k, 128k.
4. **Loudness normalization** (optional) — -14 LUFS (Spotify), -16 (Apple), -23 (broadcast).
5. **Convert All**.

How the Code Works

Builds FFmpeg arguments with optional loudnorm filter. Runs via safe_subprocess (allowlisted binary, no shell=True, mandatory timeout).

6. Tab 5: Player

Purpose

Full audio player with waveform, EQ spectrum, playlists, A-B looping.

How to Use

1. **Load a track** — File browser or drag-and-drop.
2. **Playback** — Play/Pause, Prev, Next. Seek by clicking waveform.
3. **Volume** — 0-100% slider.
4. **A-B Loop** — Set A marker, Set B marker. Playback loops between them.
5. **Playlist** — Add files, import M3U, export JSON for sharing.

Waveform

Generated in a background thread. Colors: cyan=quiet, lime=medium, yellow=loud, orange/red=peaks.

EQ Spectrum

32-bar visualization with peak hold and smooth decay animation.

Advanced

- **Crossfade** — 0-5000ms for gapless transitions
- **JSON Playlists** — Share with metadata (title, artist, BPM, key)
- **Up Next** — Shows next track

How the Code Works

Uses `_audio` singleton (pyglet) for playback. Waveform generation in thread with `threading.Lock`. Position polls every 500ms. A-B loop enforced in `_upd_pos()` — seeks to A when position exceeds B.

7. Tab 6: Analyze

Purpose

Audio analysis: BPM, key, Camelot, loudness (LUFS), track identification.

How to Use

1. **Browse** for an audio file.
2. **Analyze All** — BPM, key, loudness, waveform.
3. **Results** — BPM, Key, Camelot, LUFS, True Peak, Duration, SR, Size.
4. **Identify** — Shazam, MusicBrainz, Chromaprint, Apple Music.
5. **Write Tags** — Embed metadata into the file.
6. **Export** — JSON, CSV, or PNG image card.

Track ID Services

- **Shazam Audio ID** — 10-second audio fingerprint
- **Chromaprint/AcoustID** — Open-source fingerprinting
- **MusicBrainz** — Open music database
- **Apple Music** — Apple catalog search

Loudness Targeting

Platform presets: Spotify -14, YouTube -13, Apple -16, CD -9, Club -6, Podcast -19. 2-pass FFmpeg loudnorm.

DJ Integration

- Serato tag writing
- FL Studio file opener

How the Code Works

BPM/key via librosa `beat.beat_track()` and chroma features. Loudness via pyloudnorm. Each ID service runs in its own thread. Tag writing via mutagen (ID3, Vorbis, MP4, WAV).

8. Tab 7: Stems

Purpose

AI stem separation using Meta's Demucs.

How to Use

1. **Select audio file**.
2. **Choose model**: htdemucs, htdemucs_ft, htdemucs_6s, mdx_extra_q.
3. **Choose mode**: All Stems, Vocals Only, Drums Only, Bass Only.
4. **Split Stems** — Wait for processing.
5. **After**: Open Output, FL Studio Export, Create FLP Project, Open in FL Studio.

Models

- `htdemucs` — Default, good balance
- `htdemucs_ft` — Fine-tuned, better quality
- `htdemucs_6s` — 6 stems (adds piano + guitar)
- `mdx_extra_q` — MDX architecture

Performance

- First run downloads model (~200MB)
- GPU: 5-10x faster with CUDA PyTorch
- CPU: ~3-5 min for a 4-min track
- Batch mode for multiple files

9. Tab 8: Effects

Purpose

Audio effects chains using Spotify's pedalboard library.

How to Use

1. **Load audio file**.
2. **Add effects**: Compressor, Reverb, Delay, Distortion, Gain, NoiseGate, HPF, LPF, Shelving, Chorus, Phaser.
3. **Edit parameters** via dialog with sliders.
4. **Preview** — 5-second processed preview.
5. **Apply** — Process full file.

Advanced

- **VST3 Plugins** — Load .vst3 files from your system
- **Custom Plugins** — Hash-approved plugins from `~/.limewire/plugins/`
- **Presets** — Save/load chain configs as JSON
- **Undo/Redo** — 30-entry undo stack

How the Code Works

Uses pedalboard's `Pedalboard` class. `_build_board()` instantiates effect objects. VST3 via `pedalboard.load_plugin()`. Processing in background thread.

10. Tab 9: Discovery

Purpose

Library scanner with BPM/key caching, harmonic mixing, smart playlists.

How to Use

1. **Select folder**, click Scan Library. Recursively finds audio files.
2. **Analysis** — BPM/key via librosa. Cached in `~/.limewire_analysis_cache.json`.
3. **Find Compatible** — Camelot wheel harmonic matches.
4. **Generate DJ Playlist** — Automatic harmonic progression.

Smart Playlists

- **Energy Filter**: Low (<100), Medium (100-130), High (130-160), Very High (160+)
- **Sort Modes**: Harmonic Flow, BPM Ramp Up/Down, Key Groups

Export

- M3U with duration metadata
- CSV with BPM, Key, Camelot
- Send to Player tab

How the Code Works

`ThreadPoolExecutor(4)` for parallel analysis. Cache keyed by filepath. Harmonic matching uses Camelot rules: same number, +/-1, A/B swap. Greedy algorithm for playlist generation.

11. Tab 10: Samples

Purpose

Browse and download royalty-free samples from Freesound.org.

How to Use

1. **Enter Freesound API key** (free at freesound.org).
2. **Search** — Type query, press Enter.
3. **Filter** by duration and sort order.
4. **Preview** — Click to hear.
5. **Download** — Save preview MP3.
6. **Open in Browser** — Full quality on Freesound.

12. Tab 11: Editor

Purpose

Non-destructive audio editor with waveform selection, zoom, and operations.

How to Use

1. **Load a file**.
2. **Select region** — Click and drag on waveform.
3. **Zoom** — Mousewheel (up to 32x). Shift+scroll for horizontal pan.
4. **Minimap** — Full waveform overview at bottom with viewport indicator.
5. **Operations**: Trim, Cut, Fade In/Out, Normalize, Reverse, Silence.
6. **Undo/Redo** for all operations.
7. **Merge** — Combine multiple files.
8. **Export** in chosen format.

Snap to Zero-Crossing

Click "Snap Zero-X" for click-free cuts at waveform zero points.

How the Code Works

Uses pydub for audio operations. Canvas-based waveform with pixel-level zoom. Minimap is a separate canvas. Spectral centroid coloring via librosa for frequency-based colors.

13. Tab 12: Recorder

Purpose

Record from microphone with VU meter, live waveform, and Whisper transcription.

How to Use

1. **Select input device** from dropdown.
2. **Record** — Elapsed time shown in red.
3. **VU meter** — Green/yellow/red levels with peak hold.
4. **Stop** recording.
5. **Play back** to review.
6. **Transcribe** — Select Whisper model and language. Text appears in display.
7. **Export SRT** for timestamped subtitles.
8. **Save** — WAV, MP3, FLAC, or OGG.

How the Code Works

sounddevice async callback collects float32 frames in a locked buffer. VU is RMS in dB with peak decay. Whisper runs in background thread. SRT uses Whisper segment timestamps.

14. Tab 13: Spectrogram

Purpose

Spectral visualizations of audio files.

How to Use

1. **Load audio**.
2. **Select type**: Linear STFT, Mel, CQT.
3. **Set FFT size**: 512, 1024, 2048, 4096.
4. **Choose colormap**: viridis, magma, plasma, inferno.
5. **Generate** — Spectrogram displayed.
6. **Hover** — Shows time and frequency at cursor position.
7. **Save** as PNG/JPEG.

How the Code Works

librosa for STFT/mel/CQT computation. Amplitude to dB. Matplotlib colormaps via lookup table. PIL Image displayed on Canvas via ImageTk.

15. Tab 14: Pitch/Time

Purpose

Pitch shift and time stretch without affecting the other dimension. Vocal isolation.

Pitch Shifting

1. Load file, Detect BPM/Key.
2. Semitone slider (-12 to +12) or target key dropdown.
3. Click Shift Pitch.

Time Stretching

1. Set tempo rate (0.5x-2.0x) or target BPM with Calc Rate.
2. Click Stretch.

Vocal Isolation

1. Mode: Vocals Only, Instrumental, Full 4-Stem.
2. Model selection, then Process (uses Demucs).

How the Code Works

pyrubberband for pitch/time changes. librosa for BPM/key detection. Demucs for vocal isolation. All in background threads.

16. Tab 15: Remixer

Purpose

Mix Demucs stems with per-stem volume, pan, mute, solo.

How to Use

1. **Browse** to Demucs output folder.
2. **Channel strips** appear per stem: Mute (M), Solo (S), Volume (0-150%), Pan (L-C-R).
3. **Master volume** (0-150%).
4. **Preview Mix** — Hear current mix.
5. **Export Remix** — Save as WAV/MP3/FLAC.

MIDI Support

MIDI Learn maps hardware CC controllers to volume faders (requires mido).

How the Code Works

pydub AudioSegments per stem. `_mix_stems()` applies gain, pan, mute/solo, then overlays. MIDI via mido callback API.

17. Tab 16: Batch Processor

Purpose

Bulk operations on multiple audio files.

How to Use

1. **Add Files** or **Add Folder**.
2. **Select operations** (any combination):
    - Normalize LUFS (target value)
    - Convert Format
    - Fade In/Out (ms duration)
    - Trim Silence
    - Strip Metadata
3. **Process All**.

How the Code Works

Per-file pipeline: load with pydub, apply operations in sequence, save to output folder. Background thread.

18. Tab 17: Scheduler

Purpose

Schedule downloads for future times.

How to Use

1. Enter URL, set date/time (YYYY-MM-DD HH:MM), format, save location.
2. Add Schedule — Job appears with clock icon.
3. At scheduled time, auto-downloads in background.

Status Icons

- Clock = Pending
- Spinner = Downloading
- Checkmark = Done
- X = Failed

How the Code Works

Jobs stored in `~/.limewire_schedule.json`. Background thread polls every 30s. Due jobs trigger yt-dlp download.

19. Tab 18: History

Purpose

Complete download log with search, redownload, batch rename.

How to Use

1. **Browse history** — Status, title, source, format, date, filename.
2. **Search** — Real-time filter by title/source.
3. **Redownload** — Opens Search tab with URL.
4. **Open File** — Opens folder.
5. **Batch Rename** — Pattern template: `{title}`, `{artist}`, `{bpm}`, `{key}`, `{date}`, `{n}`, `{ext}`.

How the Code Works

History loaded from JSON. Filter uses case-insensitive substring matching. Batch rename reads ID3 tags + analysis cache. Capped at 300 entries.

20. Tab 19: Cover Art

Purpose

View, add, fetch, and manage album artwork.

How to Use

1. **Load audio** — Existing art displayed.
2. **Browse** — Select local image (PNG/JPG).
3. **Fetch** from iTunes or MusicBrainz.
4. **Apply** — Embed artwork.
5. **Remove** — Strip cover art.

How the Code Works

mutagen for extract/embed. MP3 uses ID3 APIC frames. FLAC uses picture interface. M4A/MP4 uses "covr" atom. iTunes Search API and Cover Art Archive for fetching.

21. Tab 20: Settings

Purpose

Configure preferences, theme, integrations.

Settings

- **Theme** — 13 built-in, live switching
- **Download Folder** — Default save location
- **Proxy** — HTTP/SOCKS
- **Rate Limit** — Max download speed
- **Clipboard Watch** — Auto-detect URLs
- **Discord RPC** — Show playing track in Discord
- **Language** — 6 languages
- **Worker Threads** — Parallel batch workers

How the Code Works

Reads/writes `app.settings` persisted in `~/.limewire_settings.json`. Theme changes call `apply_theme()` which updates `T` namespace, then `_reconfig_all()` walks the widget tree.

22. Theme System

13 Built-in Themes

Theme	Style
LiveWire (default)	Electric cyan, dark navy
Light	Warm neutrals, green accents
Dark	Rich dark, green accents
Modern	GitHub-inspired
Synthwave	Neon pink/purple retro
Dracula	Popular dev theme
Catppuccin	Soft pastels
Tokyo Night	Blue-tinted calm
Spotify	Green on dark
Classic	Lime green nostalgia
Nord	Arctic blue-grey
Gruvbox	Warm retro
High Contrast	Accessibility

Switching

- Settings tab dropdown
- Tools > Cycle Theme
- Ctrl+K > type "theme"

Community Themes

JSON files via Tools > Load Community Theme. Unknown keys stripped, hex colors validated.

Internal Architecture

Mutable namespace `T` — all modules use `T.BG`, `T.TEXT`, etc. Attribute lookups always return current values. `apply_theme()` sets attributes on the shared object. `_reconfig_all()` walks widget tree for immediate update.

23. Keyboard Shortcuts

Shortcut	Action
Ctrl+K	Command Palette
Ctrl+D	Download URL
Ctrl+O	Open downloads folder
Space	Play / Pause
Ctrl+Right	Next track
Ctrl+Left	Previous track
Ctrl+Up/Down	Volume
Shift+Escape	Quick close
Ctrl+?	Shortcuts dialog

Command Palette (Ctrl+K)

Fuzzy search across: 20 tab names, download history, library tracks, commands.

24. Plugin System

Creating a Plugin

Place a `.py` file in `~/.limewire/plugins/` :

```
from limewire.services.plugins import PluginBase

class MyEffect(PluginBase):
    name = "My Custom Effect"
    parameters = {
        "intensity": {
            "type": "float",
            "min": 0, "max": 1,
            "default": 0.5
        }
    }

    def process(self, audio_data, sr, **params):
        return audio_data * params.get("intensity", 0.5)
```

Hash-Based Trust

1. **Discovery** — Scans folder, computes SHA-256 hashes
2. **Approval** — You explicitly approve by hash
3. **Auto-Revoke** — File change invalidates trust
4. **Loading** — Only matching hashes are executed

25. Security Architecture

Path Safety (safe_paths.py)

- Writes confined to ~/.limewire, ~/Downloads, ~/Music, ~/Documents, temp
- Path traversal blocked (../ resolved)
- Filename sanitization: strips separators, control chars, Windows reserved names
- Atomic writes: .tmp then os.replace()
- Symlink resolution before checks

Subprocess Allowlist (safe_subprocess.py)

- Only ffmpeg, ffprobe, yt-dlp permitted
- No shell=True ever
- Mandatory timeouts (300s default, 600s for FFmpeg)
- Audit logging of all invocations

JSON Validation (safe_json.py)

- 5 MB size limit
- 10-level depth limit
- Key allowlists for themes/settings
- Hex color validation for themes

Plugin Trust (plugin_policy.py)

- SHA-256 verification before loading
- No auto-execution on discovery
- Auto-revoke on file modification

26. Code Architecture

Package Structure

```
limewire/
  __init__.py       — Version
  __main__.py       — Entry point
  app.py            — Main App(tk.Tk)
  core/             — Theme, config, constants, deps
  i18n/             — 6 languages
  utils/            — Helpers
  services/         — Analysis, metadata, plugins
  security/         — 4 safety modules
  ui/               — Widgets, styles, tooltips, toasts
  pages/            — 20 tab classes
```

Dependency Flow

```
core/ <-- utils/ <-- services/ <-- ui/ <-- pages/ <-- app.py
```

Pages get `app` via constructor — never import app.py (no circular imports).

Threading

All I/O ops run in background threads. UI updates via `widget.after(0, cb)`. Locks on shared state (waveform, recorder, scheduler).

Test Suite

185 tests via pytest covering security, core, utilities, and services. Run: `python -m pytest tests/ -v`

27. Troubleshooting

Issue	Solution
FFmpeg not found	`winget install ffmpeg`
Module not found	Check status bar, pip install
Downloads fail	`pip install -U yt-dlp`
Demucs slow	PyTorch with CUDA
Whisper fails	`pip install openai-whisper`
Plugin not loading	Approve hash in Settings
Crash on startup	Check `~/.limewire_crash.log`
Config corrupt	Delete `~/.limewire_*.json`
Import error	`python -m pytest tests/`

Log Files

- **Crash**: `~/.limewire_crash.log`
- **App**: Console via Python logging

Support

GitHub Issues: https://github.com/Ccwilliams314/LimeWire/issues

---

LimeWire Studio Edition v2.0.2

MIT License